

Oefeningen Components

Accepteer de [GitHub Classroom Assignment](#) om te starten met de oefeningen.

CSS best practice

Tijdens het maken van de oefeningen zul je in de startbestanden een `styles.css` file aantreffen. CSS die enkel nodig is voor een specifiek Vue component of view, hoort niet in deze gedeelde CSS-file thuis. Verplaats deze CSS naar het desbetreffende component of view en maak het scoped door het `<style>` blok te voorzien van het `scoped` attribuut.

```
<style scoped>
  /* je component-specifieke CSS hier */
</style>
```

vue

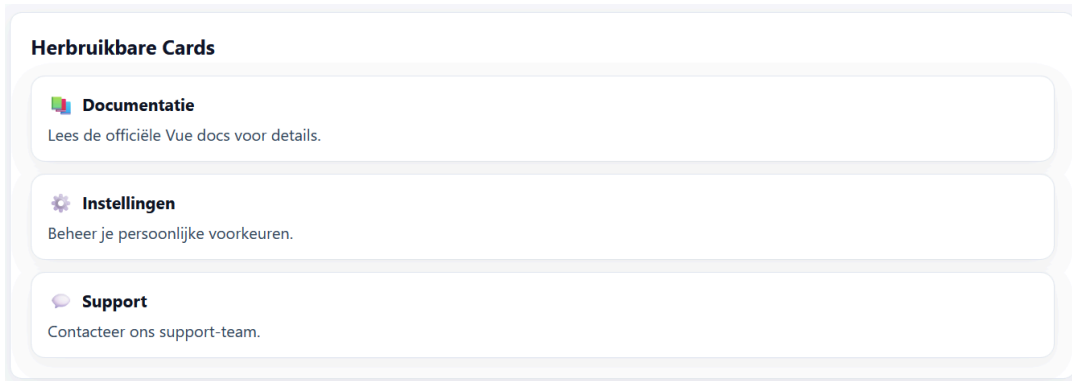
Vue 3 script template style

Wanneer een opdracht een andere volgorde heeft, bijvoorbeeld `<template>` `<script><style>`, dan refactor je dit naar `<script><template><style>`.

De volgorde is in principe niet belangrijk, maar we volgen de conventie van `<script>` bovenaan, gevolgd door `<template>` en `<style>` onderaan. Dit zorgt voor een consistente structuur in alle componenten.

Herbruikbare card component met props

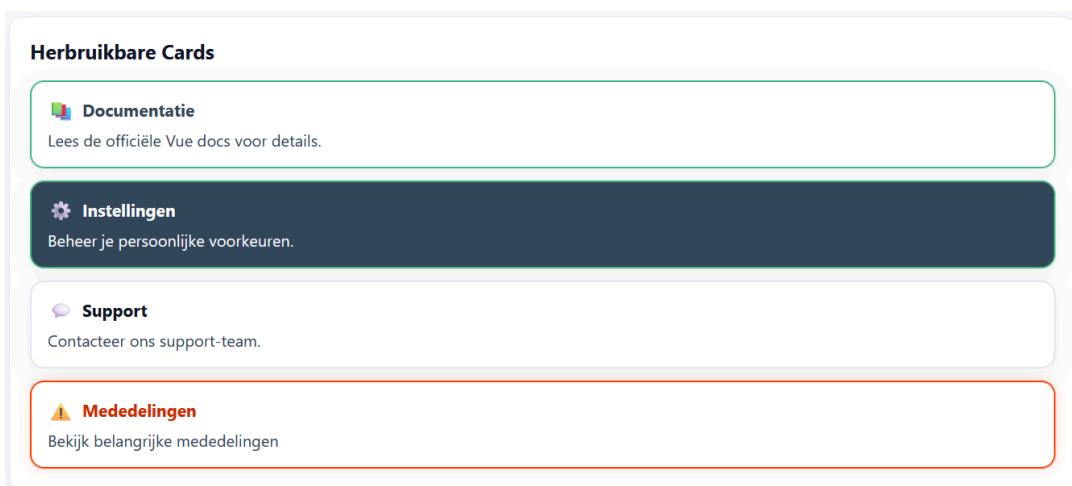
In de view staan drie **bijna identieke** card-blokken. Refactor deze naar één `CardItem` component met props: `title` (verplicht), `description`, en `icon` (default: 📄). Gebruik de component in de view.



Uitbreiding 1

Zorg ervoor dat je ook een thema kan meegeven aan de component. Afhankelijk van de `theme`-prop zal de card in een bepaalde stijl weergegeven worden.

De drie thema's zijn: `primary`, `secondary`, `important`. Wanneer geen thema meegegeven wordt, krijgt de card de standaard opmaak zoals hierboven. De **Support**-card krijgt de standaard opmaak.



TIP

In de [documentatie](#) kan je lezen hoe je conditioneel CSS-klassen kan toevoegen.

Gebruik een [computed property](#) om in je component de toe te passen CSS-klasse te bepalen.

Uitbreiding 2

Update de component zodat deze een `slot` krijgt waardoor je de beschrijving ook kan meegeven als `textContent` vanuit de parent component.

Bekijk de [documentatie](#) van `slots` om je hierbij te helpen.

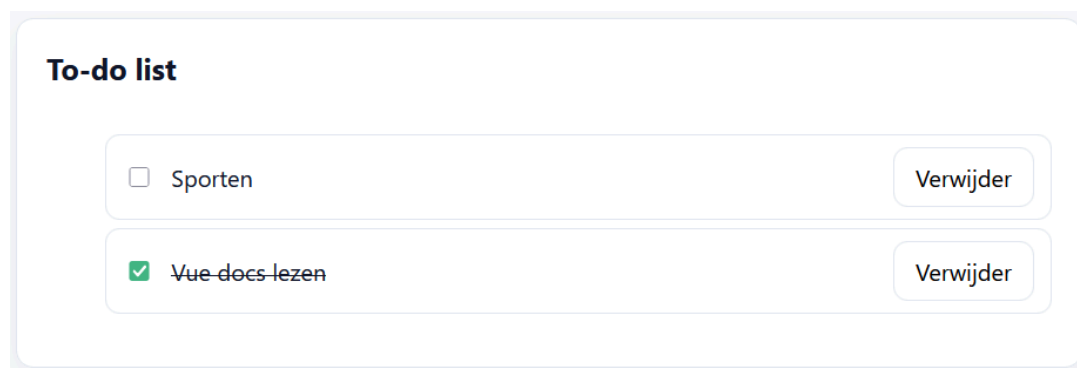
Voorbeeld:

```
<CardItem title="Support" icon="💬">Contacteer ons  
support-team.</CardItem>
```

vue

To-do list

Bestand: `ToDoView.vue`



Refactor de herhalende list items in het startbestand naar een `TodoItem` component.

De component:

- Props: `todo: { id, title, done }`

- Emits: `toggle(id)`, `remove(id)`

Gebruik `v-for` in de view om een `ToDoItem` component te renderen voor elke todo en zorg dat toggling en verwijderen werken via events.

Tips

- Valideer dat de `todo` prop de juiste velden bevat d.m.v. een validator.
- Toon afgewerkte items met `text-decoration: line-through`.

Movies & Reviews

Bouw een kleine filmlijst waarin de **parent** een array met films beheert en een **child** per film toont en het **toevoegen van reviews** mogelijk maakt. De reviews worden **in de parent** bewaard in de `movies` array.

Movies & Reviews

Inception (2010)

Reviews (1)

- De luidste film ooit gemaakt over mensen die een rustig dutje proberen te doen.

Schrijf een review...

Voeg review toe

Interstellar (2014)

Reviews (1)

- Hans Zimmer is op zijn orgel gaan zitten en creëerde per ongeluk een meesterwerk.

Schrijf een review...

Voeg review toe

Dune (2021)

Reviews (1)

- Een 155 minuten durende trailer voor deel twee. De beste tutorial voor zandlopen die ik ooit heb gezien.

Schrijf een review...

Voeg review toe

Functionaliteiten

- **Data in parent:** `movies: Array<{ id: number, title: string, year: number, reviews: Array<string> }>` staat in de parent.
- **Child per film:** de parent rendert een `<MovieItem>` voor elke film en geeft de data via props door.
- **Review toevoegen:** in het child kan je een review typen en versturen, het child **emit** een event met payload naar de parent.
- **Parent update:** de parent vangt `add-review` op en voegt de review toe aan de juiste film (`reviews.push(...)`).
- **Prop-standaardwaarde:** voorzie een **default value** bij een prop in het child (bv. `placeholder` voor het invoerveld) - *zoek zelf op hoe je defaults definieert.*

Componenten

- `views/MovieList.vue` : bevat `movies` (reactieve array), rendert `MovieItem` met props en vangt events op.
- `components/MovieItem.vue` : toont filmgegevens en bestaande reviews (gelezen uit props) en bevat een invoerformulier om een review toe te voegen. Emit `add-review` met payload `{ id, text }`.

Uitbreiding

- Validatie: lege reviews niet toevoegen en toon een foutmelding.
- Toon een teller bij elke film: `n` reviews.
- Maak een `minLength` prop met **standaardwaarde** (bv. 3) en disable de verzendknop tot de lengte is bereikt.

Pagination & Products

Je bouwt een **Pagination**-component en een **productenoverzicht**. De **parent** (PaginationView.vue) heeft een lijst producten en bepaalt hoeveel producten per pagina (**instelbaar**) worden getoond. Voorbeeld: **20 producten** en **6 per pagina** resulteren in **4 pagina's**.

Pagination & Products	
Per pagina	6
Producten 7–12 van 24	
Notitieboek	€8.99
Waterfles	€14.50
Cap	€16.00
Sokken	€9.95
Poster A3	€6.50
USB-C kabel	€11.00
Eerste	Vorige
1	2
3	4
Volgende	Laatste

Functionaliteiten

- **Props:** `modelValue` (huidige pagina, 1-indexed), `total` (totaal aantal items), `pageSize` (aantal per pagina, met standaardwaarde).
- **Navigatie:** eerste/vorige/volgende/laatste knoppen, render paginanummers.
- **Events:** `update:modelValue` (voor `v-model`) en `navigate` met payload `{ from, to }`.
- **Afgeleiden:** `totalPages = ceil(total / pageSize)`.
- **List:** toon een lijst met producten van de huidige pagina. Indien `pageSize=6` en `products.length=20` dan zijn er 4 pagina's met telkens 6/6/6/2 items.

Componenten

- `components/Pagination.vue`: de herbruikbare paginatie met `v-model` en `navigate`-event.
- `PaginationView.vue`: bevat de array `products`, beheert `page` en `pageSize`, toont het juiste slice van producten, en gebruikt `<Pagination />` met `:total="products.length"`.
- `data/products.js`: exporteert een array met minstens 20 producten `[{ id, name, price }]`.

Uitbreiding

- **PageSize-switch:** voeg een select toe (bv. 6, 9, 12) en reset de pagina naar 1 wanneer `pageSize` wijzigt.
- **Statusbalk:** toon "Producten X–Y van Z".

Pokédex Vue Edition

Over de verschillende hoofdstukken heen wordt een Pokédex-applicatie gemaakt. Elk hoofdstuk gebruikt de nieuwe kennis om de applicatie verder uit te breiden.

Accepteer de [GitHub Classroom Assignment](#) om te starten met de oefening.

De volledige opdracht is uitgewerkt in de README.md van de assignment.

Er wordt aangeraden om via `branches` te werken, elk onderdeel kan in een aparte branch gemaakt worden. De branch kan vervolgens via een Pull Request klaargezet worden voor feedback. Na feedback en goedkeuring kan de branch gemerged worden in `main` en kan de volgende oefening gestart worden in een nieuwe branch.

Verwerk voor dit hoofdstuk de onderdelen:

- Components

Previous page
[Theorie](#)

Next page
[Theorie](#)

